

Micro proyecto 1 — Generación de bocetos (Sketch-RNN)

Curso Modelos Avanzados de Procesamiento de Lenguaje Natural

1 Introducción

En este documento se describe de manera completa el micro proyecto 1 del curso, el cual comprende el contenido de la semana 1 y 2. Además de conocer el propósito del proyecto, también se presenta la rúbrica con la que se realizará la evaluación del mismo. El documento está estructurado de la siguiente forma:

Contents

1 Introducción **1**

2 Resultados de Aprendizaje **1**

3 Descripción del micro proyecto **2**
3.1 Detalles del dataset . . . 3
3.2 Límites obligatorios sobre los datos de entrenamiento . . . 4

4 Rúbrica de la nota del micro proyecto **4**
4.1 Arquitectura seleccionada . . . 4
4.2 Implementación del modelo . . . 4
4.3 Entrenamiento del modelo . . . 4
4.4 Resultados obtenidos . . . 5
4.5 Cumplimiento de requisitos de la actividad . . . 5

5 Entregables en Coursera **5**
5.1 Estructura obligatoria del Jupyter Notebook . . . 6

2 Resultados de Aprendizaje

Con esta actividad se busca que el estudiante pueda poner en práctica el uso de modelos de redes neuronales para trabajar con datos secuenciales. Tras realizar esta actividad se espera que el estudiante esté en capacidad de:

1. Estructurar datos secuenciales de forma que puedan ser usados por modelos de redes neuronales.
2. Desarrollar soluciones de machine learning para datos secuenciales en distintos dominios.
3. Entrenar modelos de machine learning con múltiples salidas.

4. Entrenar modelos de machine learning con una función de costo compuesta.

5. Utilizar el modelo entrenado para generar nuevas secuencias.

3 Descripción del micro proyecto

Para este micro proyecto, usted dispone del dataset *Quick, Draw!*, una colección abierta de aproximadamente 50 millones de bocetos hechos a mano por usuarios de todo el mundo, organizados en 345 categorías de objetos (gato, bicicleta, manzana, casa, etc.). Cada boceto es un **dibujo vectorial**: una secuencia de movimientos del lápiz sobre un plano. El objetivo del proyecto es que usted entrene un modelo con los bocetos de **una única categoría** que tenga la capacidad de **generar nuevos bocetos** en el estilo de esa categoría.

Elija una de las categorías disponibles y utilice los bocetos de esa **única categoría** como datos de entrenamiento y validación, sujeto a los dos límites **obligatorios** que se describen a continuación:

- Debe tomar un **máximo de 20.000 bocetos** de la categoría seleccionada.
- Cada boceto debe truncarse a un **máximo de 200 puntos** por boceto.

Ambos límites son obligatorios. Los datos preprocesados para modelos recurrentes (versión *sketch-rnn*) se obtienen de la siguiente forma. Cada categoría está almacenada en su propio archivo `.npz`, que ya viene dividido en conjuntos de entrenamiento, validación y prueba:

```
import numpy as np
import urllib.request

# Elija una categoria del dataset (por ejemplo: "cat", "bicycle", "apple"...)
category = "cat"
url = f"https://storage.googleapis.com/quickdraw_dataset/sketchrnn/{category}.npz"
urllib.request.urlretrieve(url, f"{category}.npz")

# En Python 3 se debe cargar con encoding='latin1' y allow_pickle=True
data = np.load(f"{category}.npz", encoding='latin1', allow_pickle=True)
train_sketches = data['train'] # arreglo de dibujos de entrenamiento
valid_sketches = data['valid'] # arreglo de dibujos de validacion
test_sketches = data['test'] # arreglo de dibujos de prueba

# Cada dibujo es una secuencia de puntos en formato "stroke-3": (dx, dy, pen_lift)
sketch = train_sketches[0]
print('Forma del dibujo:', sketch.shape) # (N_puntos, 3)
print(sketch[:5])
```

Cada boceto está representado como una secuencia de puntos del lápiz. En el formato original (*stroke-3*) cada punto se compone de tres valores: $(\Delta x, \Delta y, \text{pen_lift})$. A partir de ellos, para el entrenamiento del modelo use las siguientes características para cada punto en la secuencia:

- Δx **(delta_x):** característica numérica que representa el desplazamiento horizontal del lápiz respecto al punto anterior.
- Δy **(delta_y):** característica numérica que representa el desplazamiento vertical del lápiz respecto al punto anterior.
- **pen_state (estado del lápiz):** variable **categórica** con 3 posibles estados. Se obtiene a partir del valor `pen_lift` del formato *stroke-3* (convirtiéndolo al formato *stroke-5*):
    1. El lápiz está tocando el papel: se dibujará una línea uniendo el punto actual con el siguiente.
    2. El lápiz se levanta después del punto actual: no se dibujará una línea hacia el siguiente punto (fin de un trazo).
    3. El dibujo ha terminado: este punto y los siguientes no se dibujan.

Recomendación: dado que Δx y Δy son valores numéricos con distinta escala, se recomienda normalizarlos (por ejemplo, dividiéndolos por su desviación estándar) antes de entregarlos al modelo, y revertir la normalización en el momento de la generación.

Cree un modelo que haga uso de las características mencionadas de puntos anteriores para generar el siguiente punto, el cual está compuesto por las mismas características, por lo que su modelo debe

tener la capacidad de generar **varias salidas**: dos salidas numéricas (Δx y Δy) y una salida categórica (pen_state). Vea la Figura 1 para ver el detalle de las entradas y salidas del modelo.

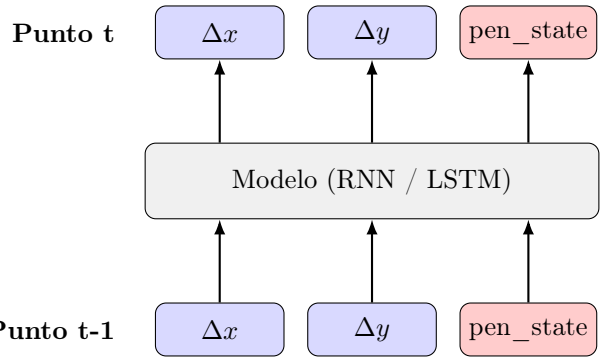


Figure 1: Diagrama con los datos de entrada y de salida del modelo que debe implementar. Su modelo recibirá como entrada las 3 características de los puntos que recibe como contexto ($\text{Punto}_{t-1}, \text{Punto}_{t-2}, \ldots, \text{Punto}_{t-c}$, donde c es el tamaño del contexto), y predice las mismas 3 características para el punto en el tiempo t (Punto_t). Las salidas Δx y Δy son numéricas (regresión) y pen_state es categórica (clasificación de 3 clases).

Finalmente, para poder apreciar la secuencia de puntos generada por su modelo, debe renderizar el boceto como una imagen. Para ello, se transforman los desplazamientos $(\Delta x, \Delta y)$ en posiciones absolutas (mediante una suma acumulada) y se dibuja cada trazo deteniéndose cuando el estado del lápiz indica el fin de un trazo. A continuación vea un fragmento de código guía para renderizar y guardar un boceto en formato PNG:

```
import matplotlib.pyplot as plt
import numpy as np

def render_sketch(sketch, filename):
    # sketch en formato stroke-3: arreglo (N, 3) con columnas (dx, dy, pen_lift)
    # De desplazamientos relativos a posiciones absolutas
    coords = np.cumsum(sketch[:, :2], axis=0)

    fig, ax = plt.subplots()
    start = 0
    for i in range(len(sketch)):
        # pen_lift == 1 indica el fin de un trazo
        if sketch[i, 2] == 1:
            ax.plot(coords[start:i+1, 0], -coords[start:i+1, 1], color='black', linewidth=2)
            start = i + 1
    ax.set_aspect('equal')
    ax.axis('off')
    fig.savefig(filename, bbox_inches='tight')
    plt.close(fig)

# Ejemplo: renderizar el primer dibujo de entrenamiento
render_sketch(train_sketches[0], 'ejemplo.png')
```

Para realizar la generación, parta de un punto inicial (semilla) y, de forma autorregresiva, use su modelo para predecir el siguiente punto a partir de los c puntos previos. Para Δx y Δy use las salidas numéricas del modelo, y para pen_state muestree (o tome la clase más probable de) la distribución sobre los 3 estados. Detenga la generación cuando el modelo prediga el estado de *fin del dibujo* o cuando alcance un número máximo de puntos.

3.1 Detalles del dataset

El dataset *Quick, Draw!* se compone de bocetos distribuidos en 345 categorías. A continuación se listan algunas categorías a modo de ejemplo:

- cat (gato), dog (perro), apple (manzana), bicycle (bicicleta)
- house (casa), tree (árbol), car (carro), flower (flor)
- fish (pez), airplane (avión), clock (reloj), umbrella (sombrilla)

El listado completo de categorías y la documentación oficial del dataset se encuentran en el repositorio de Google *Creative Lab*: github.com/googlecreativelab/quickdraw-dataset. Cada archivo `.npz` de la versión *sketch-rnn* contiene 70.000 ejemplos de entrenamiento, 2.500 de validación y 2.500 de prueba por categoría, simplificados con el algoritmo RDP. Si desea utilizar el conjunto completo de una categoría, puede descargar el archivo con extensión `.full.npz`.

3.2 Límites obligatorios sobre los datos de entrenamiento

Para el entrenamiento del modelo usted debe respetar los siguientes dos límites, ambos **obligatorios**:

- **Máximo 20.000 bocetos** de la categoría seleccionada, contando los conjuntos de entrenamiento, validación y prueba en total.
- **Máximo 200 puntos por boceto.** Cada secuencia debe truncarse a lo sumo a 200 puntos.

Esos 20.000 bocetos deben repartirse en una división **80 / 10 / 10**, respetando los conjuntos ya provistos en el archivo `.npz`:

- **16.000 bocetos para entrenamiento** (tomados del conjunto `train`).
- **2.000 bocetos para validación** (tomados del conjunto `valid`).
- **2.000 bocetos para prueba** (tomados del conjunto `test`).

Estos límites mantienen el proyecto resoluble con recursos de cómputo modestos y aseguran tiempos de entrenamiento razonables.

4 Rúbrica de la nota del micro proyecto

La nota del micro proyecto se compone de la forma descrita en la siguiente tabla:

Item	**Porcentaje de la nota**
Arquitectura seleccionada	30%
Implementación del modelo	20%
Entrenamiento del modelo	30%
Resultados obtenidos	20%

Table 1: Composición de la nota del micro proyecto

4.1 Arquitectura seleccionada

El estudiante debe crear un diagrama que detalle la arquitectura implementada. El diagrama no tiene una forma específica definida, pero sí debe reflejar los datos de entrada que acepta, la forma en que se procesan los datos por el modelo, y sus salidas en tiempo de entrenamiento. Se evaluará adicionalmente la idoneidad de la arquitectura planteada para la solución del problema. El diagrama debe estar acompañado por una explicación detallada de la arquitectura, y la justificación de su selección.

4.2 Implementación del modelo

Se evaluará que el modelo corresponda a la arquitectura planteada, y que la implementación sea técnicamente correcta. Adicionalmente, se evaluará el grado en el que se demuestra el conocimiento sobre el uso de PyTorch, y de redes neuronales en general.

4.3 Entrenamiento del modelo

Se evaluará la implementación del bucle de entrenamiento construido, la función de pérdida utilizada (la cual se espera sea una composición de distintos objetivos de entrenamiento: una componente de regresión para Δx y Δy, y una componente de clasificación para pen_state), y la selección razonable de los hiperparámetros del modelo. Adicionalmente, se evaluará la forma en la que se usaron los datos para realizar el entrenamiento del modelo.

4.4 Resultados obtenidos

Mida qué tan bien generaliza el modelo usando el **conjunto de prueba** (los 2.000 bocetos del conjunto `test` del split obligatorio). Recuerde el papel de cada conjunto: el de validación sirve para ajustar hiperparámetros y monitorear el entrenamiento; el de prueba se usa **solo al final** para el reporte. Se evaluará qué tan bien se ajusta el modelo a los datos y qué hizo usted para evitar el sobre ajuste.
Métricas obligatorias. Sobre el conjunto de prueba, reporte una métrica por cada salida del modelo:

- **MSE** (error cuadrático medio) para las salidas numéricas Δx y Δy, es decir, la parte de regresión.
- **F1 macro** para la clasificación de `pen_state` (sus 3 estados). Como las clases están desbalanceadas, el F1 macro evita que los estados poco frecuentes queden opacados por el más común.

Reporte también el valor de la **función de pérdida** sobre el conjunto de prueba. Todas estas cantidades se obtienen comparando, punto por punto, la predicción del modelo con el valor real (predicción a un paso: para predecir cada punto se usan los puntos reales anteriores como contexto).
Cómo agregar las métricas: por punto, no por figura. Cada boceto tiene muchos puntos. Junte los puntos de *todas* las secuencias de prueba en una sola lista de pares (predicho, real) y calcule **una única métrica global** sobre esa lista: un solo MSE y un solo F1 macro. Así cada punto pesa lo mismo, sin promediar figura por figura.
Cuidado con el *padding*. Como los bocetos tienen distinta longitud pero el modelo procesa lotes de secuencias del mismo tamaño, lo usual es fijar todas a 200 puntos: se truncan las más largas y se rellenan las más cortas con **puntos de relleno** (*padding*; por ejemplo ceros o el estado de fin de dibujo). Esos puntos no existen en el dibujo real, así que debe **enmascararlos**: guarde cuántos puntos reales tiene cada secuencia y descarte el resto, de modo que el relleno no entre ni en las métricas ni en la pérdida. Si no lo hace, los resultados quedan distorsionados por puntos que no existen.
Por último, genere al menos **3 archivos PNG**, cada uno con un boceto completo producido por su modelo (hasta 200 puntos por dibujo, o menos si el modelo predice el fin del dibujo antes).

4.5 Cumplimiento de requisitos de la actividad

Recuerde que su modelo no tendrá validez si no cumple los requisitos que se mencionan a continuación. Por lo tanto, obtendrá una calificación de 0 en la actividad correspondiente al micro proyecto.

Los requisitos son los siguientes:

- Para el entrenamiento del modelo solo se deben usar los datos proporcionados (la categoría seleccionada del dataset *Quick, Draw!*), no podrá usar datos de otras fuentes.
- Debe implementar y entrenar modelos desde cero haciendo uso de PyTorch, y si lo desea de las capas disponibles en la librería como Linear, LSTM, RNN, Embedding, entre otras.
- El código enviado debe poder ser ejecutado por el equipo evaluador, y el código debe ser completamente de su autoría.

5 Entregables en Coursera

En la semana 2 del curso se habilitará una actividad en donde usted debe subir los siguientes entregables:

1. Jupyter Notebook que se utilizó para realizar la implementación y el entrenamiento del modelo. La imagen del diagrama junto con cualquier texto debe estar embebido dentro del Jupyter notebook.
2. La imagen con el diagrama también debe adjuntarla por aparte para asegurarse de que el equipo evaluador pueda visualizarla correctamente.
3. Tres archivos tipo PNG, en donde cada archivo debe contener un boceto completo generado por su modelo.

5.1 Estructura obligatoria del Jupyter Notebook

El Jupyter Notebook debe organizarse *obligatoriamente* en las siguientes secciones, en este mismo orden y con estos mismos títulos. El notebook debe ser autocontenido y ejecutable de principio a fin (*Run All*). Entre paréntesis se indica el ítem de la rúbrica al que contribuye cada sección:

1. **Portada e introducción.** Nombre del estudiante, **categoría seleccionada** y un resumen del proyecto. Debe incluir el **diagrama de la arquitectura** embebido, junto con su explicación detallada y la justificación de su selección. *(Arquitectura seleccionada)*
2. **Librerías.** Importación de todas las dependencias utilizadas.
3. **Lectura del dataset y exploración.** Descarga del archivo `.npz` y exploración de los datos (por ejemplo, distribución de longitudes y bocetos de ejemplo renderizados).
4. **Preprocesamiento.** Conversión de *stroke-3* a *stroke-5*, normalización de Δx y Δy, truncado y relleno (*padding*) a 200 puntos, construcción de la máscara y armado de los pares entrada $\rightarrow$ objetivo. *(Entrenamiento del modelo: uso de los datos)*
5. **Split train / validación / prueba.** Aplicación del split obligatorio (16.000 / 2.000 / 2.000) sobre los conjuntos provistos en el `.npz`. *(Entrenamiento del modelo: uso de los datos)*
6. **Arquitectura y creación del modelo.** Implementación en PyTorch del modelo, coherente con el diagrama de la portada. *(Implementación del modelo)*
7. **Entrenamiento del modelo.** Bucle de entrenamiento, función de pérdida compuesta (regresión + clasificación), selección de hiperparámetros y curvas de entrenamiento y validación. *(Entrenamiento del modelo)*
8. **Evaluación del modelo.** Cálculo del MSE y del F1 macro sobre el **conjunto de prueba** (con la máscara aplicada), y del valor de la función de pérdida. *(Resultados obtenidos)*
9. **Generación de ejemplos.** Generación autorregresiva de bocetos y creación de los **al menos 3 archivos PNG**. *(Resultados obtenidos)*
10. **Conclusiones.** Discusión de los resultados, del sobre ajuste y de las limitaciones del modelo.